



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

GOOGLE PUB/SUB PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Messaging & Streaming

TOTAL: 10 QUESTIONS

Q1 What is Google Pub/Sub and what problem in Messaging & Streaming does it solve?

Threat Model

Q6 How does Google Pub/Sub handle reliability and high availability?

Observability

Q2 What are the core components or concepts in Google Pub/Sub?

Architecture

Q7 What observability does Google Pub/Sub provide out of the box?

Lifecycle

Q3 How does Google Pub/Sub compare to its main alternatives?

Configuration

Q8 What are common performance bottlenecks in Google Pub/Sub?

Compliance

Q4 How do you get started with Google Pub/Sub for a new project?

Hardening

Q9 What security best practices apply when running Google Pub/Sub?

Testing

Q5 What configuration options most affect Google Pub/Sub's behavior in production?

SSO / IdP

Q10 What mistakes do teams commonly make when adopting Google Pub/Sub?

Tuning



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Google Pub/Sub is a tool or

Mitigation

Google Pub/Sub is a tool or platform that automates or simplifies a specific concern in the Messaging & Streaming domain, reducing manual effort and operational complexity for engineering teams.

A6 Google Pub/Sub provides HA through leader

Observability

Google Pub/Sub provides HA through leader election, replication, or stateless horizontal scaling. Deploy at least two instances across availability zones and configure health checks for automatic failover.

A2 Google Pub/Sub has key building blocks

Primitives

Google Pub/Sub has key building blocks that work together: a configuration layer defines intent, a runtime layer enforces it, and an API or CLI layer allows operators to manage and observe the system.

A7 Google Pub/Sub exposes metrics (Prometheus

Automation

Google Pub/Sub exposes metrics (Prometheus endpoint or built-in dashboard), structured logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

A3 Google Pub/Sub trades off simplicity against

Hardening

Google Pub/Sub trades off simplicity against feature richness differently than its alternatives. Choose Google Pub/Sub when its specific strengths performance, ecosystem, or ease of use align with your team's primary constraints.

A8 Performance bottlenecks typically stem from

Governance

Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

A4 Install Google Pub/Sub via its package

Gotchas

Install Google Pub/Sub via its package manager or binary release. Follow the quickstart to initialize a project, configure the minimal required settings, and run the default command to verify the setup works end-to-end.

A9 Run Google Pub/Sub with the principle

Assessment

Run Google Pub/Sub with the principle of least privilege, enable TLS for all communication, use a secrets manager for credentials, and keep the software patched to the latest stable release.

A5 Production-critical settings typically include

Federation

Production-critical settings typically include concurrency limits, timeout values, retry policies, resource limits, and logging levels. Review the official production hardening guide before deploying.

A10 Common pitfalls include skipping the

Optimization

Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.